The Unseen Block: What Split-Instant Evaluation Hides in KV-Cache Placement [Experiment, Analysis & Benchmark]

Suan Lee
Semyung University
Jecheon, Republic of Korea
suanlee@semyung.ac.kr

ABSTRACT

A natural way to evaluate how a memory system for LLM agents should place KV-cache blocks across storage tiers, and the one we used for three revisions of this work, is at a split instant: observe a prefix, place the blocks that exist, and bill the result against a hindsight oracle. On two production Mooncake traces this protocol measures little. Of the oracle's advantage, 84.4% and 87.6% sits on blocks that do not yet exist at the split, so the best possible split-time policy reaches 15.6% and 12.4% of it; the share stays at 75–97% for every split from 0.3 to 0.7 and under tenfold changes to the latency and token weights, though on short traces much more is reachable. Inside that protocol a better ranker helps monotonically, but only over that sliver. In a static analysis, deciding at each block's arrival puts more within reach, but how much turns on a decision a split-time evaluation never meets: where a block never seen ranks against blocks seen once. The prefix says above them (a new block's expected reuse is 0.52 against 0.30), and ranking it there captures 2.3–4.9 times what a clairvoyant split-time policy can; ranking it behind them can leave arrival-time placement at that ceiling. Billed on the same metric, plain recency does better: a multi-tier LRU, which ranks every accessed block first, captures 84.5% and 87.7% when moving blocks is free, and once a move costs one read and one write none of the policies we replay that move blocks beats the static frequency heuristic. What split-instant evaluation hides is recency and the price of migration, not a better predictor. Reactive tiering is consistent with this: LRU, LFU and a fill-once cache reach 75–99% (toolagent) and 40–94% (conversation) of Belady's hit rate, the best adaptive policy (ARC, LeCaR, CACHEUS) beats the best of those rules, each chosen per operating point, by at most 4.6 points, and a learned replacement policy (LRB) stays below ARC, because 76–78% of blocks are never reused. We give a one-line diagnostic for the protocol failure, release an artifact that regenerates every number on CPU, and catalogue the measurement defects we made and caught, including leaks in our own arrival-time experiment that survived every number check.

PVLDB Reference Format:
Suan Lee. The Unseen Block: What Split-Instant Evaluation Hides in KV-Cache Placement. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/suanlab/stoa.

1 INTRODUCTION

Large language model (LLM) agents increasingly depend on memory that outlives a single context window—the facts they extract, the turns they have seen, the tools they have run, the summaries they have written. How well an agent answers, how fast it responds, and how much it costs are all decided by one question asked of every memory item: **in what form should it be kept, where should it live, and when should it be (re)materialized?** Concretely, each item can be stored as plaintext, a vector, a graph, a latent/KV activation, or baked into parameters; it can sit on GPU, CPU, disk, or a remote tier; and it can be written inline while serving, consolidated during an idle window, or left untouched.

Today this single decision is **fragmented across three research communities**, each of which optimizes one axis and freezes the others. Memory-operating-system work—MemOS [7], MemGPT [12]—provides the primitives to migrate an item between representations and tiers, but *decides when to migrate by hand-tuned heuristics*. Reinforcement-learned memory managers—Memory-R1 [22], Memory-as-Action [23]—learn *what content* to add, update, or delete, but operate over a single fixed representation and never choose a tier or schedule an offline write. Systems-level cache tiering—LMCache [1]—places bytes across GPU/CPU/disk/remote to minimize *miss rate or time-to-first-token*, with no notion of the downstream task utility that the memory actually serves.

Our central observation is that **these three axes are coupled and cannot be optimized separately.** Promoting an item to a latent or parameter representation is an expensive offline write, but it is then cheap and fast to serve; leaving it as plaintext on disk is cheap to write but slow to read and heavy on context tokens. The best representation depends on the tier and on how often the item will be reused; the best tier depends on the representation's serving cost and the item's size; and whether to pay the promotion cost at all depends on the budget. This is precisely a **budget-constrained scheduling problem**—and a large body of ML-for-systems work (Belady-oracle imitation for cache replacement [9], GNN schedulers for clusters [10], offline-RL cache eviction in production [4]) demonstrates that such problems are learnable.

The reason to act now is that every substrate exists but nothing orchestrates them: MemCube migration, RL memory managers, tiered KV caches, latent memory [19], and sleep-time consolidation [8] all landed by late 2025, unconnected. What is missing is a *learned scheduler* that, per item and under a shared budget, trades

accuracy against latency and cost, with **task utility—not miss rate—as the objective.**

We frame this as **STOA** (Storage & Tiered Orchestration for Agents): one budget-constrained Markov decision process over **representation × tier × timing**, per memory item, which a learned control plane *would* solve with a serve-time latency/cost/token budget (L, C, B) as its knob. We do not build or evaluate that policy. We take the question the frame invites, *is this decision learnable?*, to production KV traces, and the measurement turned out to hinge less on the model than on what the evaluation protocol lets a policy see: a block that does not exist yet.

A natural way to evaluate this decision, and the one earlier versions of this paper used, fixes a split instant: observe a prefix, build features from it, place every block, and bill the result against the future. Feature construction can only describe a block that has been seen — and on these traces 45–46% of blocks have no pre-split access at all, while the hindsight oracle they are scored against ranks them anyway. Those blocks carry **84–88% of the oracle's advantage**. A policy confined to the remainder looks flat however well it ranks, which is precisely what our earlier "ranking does not convert into placement" result measured. We retract it.

Moving the decision to each block's *arrival*, placing it the moment it first appears from the session context available at that instant, changes what is within reach in a static analysis, but only if one decision is made well: where a block never seen ranks against blocks already seen. The prefix says a newly arrived block will be reused more than a block seen once; ranking it there captures 2.3 to 4.9 times what a clairvoyant split-time policy reaches (**15.6%** and **12.4%** of the oracle's advantage, or 20.4% and 15.3% when only the tier is chosen), while ranking it behind seen-once blocks moves arrival-time placement down by 14 to 29 points and, with tier-only actions, to that ceiling. How new blocks are ranked among themselves moves the result by at most 5.3 points. But plain recency, billed on the same metric, does better: a multi-tier LRU captures 84.5% and 87.7% when moving blocks is free, and once a move costs one read and one write none of the policies we replay that move blocks beats the static frequency heuristic. What the split-instant protocol hides, then, is recency and the price of migration, not a better predictor. A synthetic control does not follow this pattern.

Reaction is competitive for a related reason. Tiering rules that forecast nothing at all — LRU, LFU and a fill-once cache — reach 75–99% of Belady's hit rate on one Mooncake trace and 40–94% on the other, and the whole adaptive-replacement family (ARC, LeCaR, CACHEUS) beats the best of them by *at most 4.6 points*. All of them adapt on a ghost hit, and 76–78% of blocks here are never reused, so most evictions are correct and emit no signal: ARC gets one on 3.5–7.3% of misses. A population of singletons starves the channel adaptive replacement runs on.

We report how much measurement discipline this took, because it is the part most likely to transfer, and because it is not a flattering story. Three claims were retracted, the last two only under an adversarial review we ran with AI reviewer agents, *after* the error catalogue below had been written: a headline that randomized sampling showed had never been measurable, the decoupling result above, and an LRB column that turned out to measure random eviction because a training buffer was truncated to its tail. Later still, after every number check passed, an audit of our own arrival-time experiment found that its training labels, its split and its belief magnitudes all used information a deployed policy would not have; correcting them withdrew four readings this paper used to make, and a later re-verification found that its order comparison had been a magnitude comparison, withdrawing two more. In every case the aggregate looked reasonable, which is the point — none of our checks asked whether a reported quantity was arithmetically able to be what it claimed, and none asked whether a stored result was older than the program that wrote it.

Contributions.

- **(C1)** A frame, not an evaluated system: agent-memory placement as one budget-constrained MDP over representation × tier × timing with a serve-time (L, C, B) knob (Section 3), and a control-plane sketch for it (Section 4). The traces carry signal for the tier axis only; the simulator also chooses representation and timing from placeholder cost tables, so we report the arrival-time result with and without those choices, offer the formulation as the frame the measurements are read against, and do not claim to have evaluated it.
- **(C2)** A timing result on production traces. A split-instant evaluation protocol makes 84–88% of a hindsight oracle's advantage unreachable, because 45–46% of blocks do not exist yet when it decides (Section 5.2); inside it, a better ranker helps monotonically but over a 12–16% sliver (Section 5.4). Deciding at each block's *arrival* captures 2.3–4.9 times the matching split-time ceiling when unseen blocks are ranked above seen-once blocks, as the prefix says they should be (Section 5.3); on the same metric plain recency captures more when migration is free, and with migration priced none of the moving policies we replay beats the static heuristic.
- **(C3)** Measurements of reactive tiering on both sources against the adaptive-replacement family (ARC, LeCaR, CACHEUS) and a reimplemented LRB — the best of them beating the best plain rule by at most 4.6 points of Belady across eight operating points — together with an eviction-signal inversion that makes a policy tuned on one source a poor default for the other, and a measured explanation: KV traces are dominated by one-time blocks, which starve the ghost-hit channel the three adaptive policies run on and leave most of LRB's training labels censored (Section 5.6).
- **(C4)** A reusable evaluation protocol: twenty-seven measurement defects that each manufactured a plausible result here, grouped into nine mechanisms, and the cheap controls that catch them — including the rule that a null is reportable only alongside the sample size that would have resolved it, and the rule that every edge of the source → artifact → figure → paper chain needs its own freshness check (Section 5.7).

We position STOA against prior work along the three axes it unifies in Table 1; no prior system occupies their intersection—deciding representation, tier, *and* timing, learned, under a shared budget, with task utility as the objective.

System	Repr.	Tier	Timing	Task util.	Budget cond.	Joint credit
MemOS [7]	○	○	○	○	○	–
Memory-R1 [22]	–	–	✓	✓	○	○
Mem-as-Act. [23]	–	–	✓	✓	○	✓†
LMCache [1]	–	✓	○	○	○	–
Cold-RL [4]	–	✓	–	○	○	✓†
Sleep-time [8]	–	–	✓	✓	○	–
STOA (proposed)	∗	∗	∗	∗	∗	∗

Table 1: No prior system decides representation, tier, *and* timing jointly—learned, under a shared budget, with task utility as the objective. ✓: learned; ○: fixed/heuristic; –: not applicable. †: credit for one axis only (content or eviction). ∗: proposed in Section 3–4 and *not evaluated in this paper* — the row states an intended design, not a measured capability, and Section 5 gives reasons to doubt parts of it.

Artifact. Every result in this paper is regenerated on CPU, without a paid API, by the released package at https://github.com/suanlab/stoa; `make verify` re-derives each one from its stored result in a few minutes; `make reproduce-full` regenerates them from the public Mooncake and kv-cache-tester traces, `make reproduce-external` the Alibaba replication, and the block-move timings need a CUDA host (`measure_tier_costs.py`).

2 MOTIVATION: WHY NOW

The substrates exist; the orchestration does not. By late 2025 every mechanism STOA needs had been built—in isolation. MemOS [7] exposes a MemCube abstraction with primitives to migrate an item between plaintext, activation, and parameter forms. LMCache [1] tiers KV-cache bytes across GPU, CPU, and remote storage. M+ [19] maintains a latent memory with a co-trained retriever, and sleep-time compute [8] shows that precomputing during idle windows cuts test-time cost by roughly 5×. RL memory managers [22, 23] learn what to write and edit. What is absent is the layer that *decides*, per item and under a shared budget, which of these mechanisms to invoke and when—a scheduler whose objective is downstream task utility rather than cache miss-rate.

A worked coupling example. Consider a single fact accessed k times over a session (placeholder costs from our simulator). As plaintext on CPU it costs nothing to write but ∼800 tokens on every serve, so its serving cost grows as $800k$ tokens. Promoted once to a latent representation it costs a one-off consolidation but serves at ≈0 tokens thereafter. For a cold fact (k=1) the promotion never pays off; for a hot fact ($k \gg 1$) it is decisively cheaper—and the crossover depends on the tier (which sets read latency), on the timing (paying the promotion online vs. amortizing it in a sleep window), and on the residual token budget B. The three axes cannot be set independently, and the right setting is data-dependent, which is why one might want to learn it.

Why a control plane, and why learned. Hand-tuned migration rules (as in today's memory-OS systems) cannot track this crossover across workloads, budgets, and backbones. But the decision is not intractable: it is a budget-constrained scheduling problem, and adjacent problems—cache replacement [4, 9] and cluster scheduling [10]—have repeatedly been shown learnable from traces. STOA frames the joint representation–tier–timing decision the same way, as a control plane that would sit above existing stores and tiers; whether it is learnable on real traces is what Section 5 measures.

3 PROBLEM FORMULATION

3.1 The per-item decision

An agent accumulates memory items $\mathcal{M} = \{m_1, \ldots, m_N\}$—facts, dialogue turns, tool results, summaries. For each item a memory system must answer three questions that are today decided separately: *representation* $\rho \in \{\text{plaintext}, \text{vector}, \text{graph}, \text{latent}, \text{parameter}\}$, the semantic form (trading fidelity, serving-token cost, write cost); *tier* $\tau \in \{\text{GPU}, \text{CPU}, \text{disk}, \text{remote}\}$, the physical location (trading read latency and capacity); and *timing* $t \in \{\text{online}, \text{sleep}, \text{noop}\}$, *when* the placement is (re)materialized.

The central claim is that these axes are **coupled**. Promoting an item to latent or parameter is an expensive offline write but then cheap and fast to serve; leaving it as plaintext on disk is cheap to write but slow to read and token-heavy. The optimal representation depends on the tier and on reuse; the optimal tier depends on the representation's size and serving cost. No axis can be optimized in isolation.

3.2 A budget-constrained MDP

We cast the orchestrator as a budget-constrained MDP solved by a learned policy π_θ.

State. For each candidate item m_i the state $s_t^{(i)}$ concatenates: (a) *semantic* features—embedding, recency, access frequency, a provenance/version tag, a predicted next-access time (a Belady-style forecast); (b) *systems* features—current representation and tier, item size, and per-tier occupancy/pressure; (c) *budget* features—residual L, C, B; and (d) *query* context. Items are the nodes of a *memory graph* (edges: co-access, entity links) perceived jointly.

Action. Each item is assigned one action $a = (\rho, \tau, t)$. The raw space has $|\rho| \cdot |\tau| \cdot |t| = 5 \cdot 4 \cdot 3 = 60$ combinations, but some are illegal and are **masked**: a parameter promotion is sleep-only (never inline online), and latent/KV memory is meaningful only on GPU/CPU. Masking removes $4 + 6 = 10$ combinations, leaving a fixed legal set of $|\mathcal{A}| = 50$ per item. Legality is defined once and is load-bearing: both the action enumeration and the policy's output distribution are defined over exactly these 50 actions.

Transition. Executing a realizes a cost tuple (lat, cost, tok, fresh) from a transition model whose cost tables are placeholders (Section 5.7). Timing sets the economics: a sleep action amortizes the one-off promotion cost off the serving path; a noop serves the item from its current form at no write cost.

Objective. We maximize expected task utility subject to serve-time budgets:

$$\max_\theta \ \mathbb{E}\Big[\textstyle\sum_t U(a_t)\Big] \ \text{ s.t. } \ \text{Latency} \le L,\ \text{Cost} \le C,\ \text{Tokens} \le B, \quad (1)$$

where U is a downstream reward (QA accuracy, or a step-level process reward) realized only when the item is *used* to answer a

query—many steps after placement. We relax the constraints with learned duals:

$$r_t = U - \lambda_L \,\text{lat} - \lambda_C \,\text{cost} - \lambda_B \,\text{tok} + \beta \,\text{fresh}. \tag{2}$$

The operator sets (L, C, B) at serving time and the policy adapts, so a *single control knob* moves the system along its accuracy–latency–cost frontier—the key operational difference from a statically configured memory system. We do not evaluate it here.

3.3 A factored policy over the memory graph

A joint head over 50 actions per item, across many items, is combinatorial. We factor the policy over a shared trunk, $\pi_\theta(a \mid s) \propto \pi_{\text{rep}}(\rho)\, \pi_{\text{tier}}(\tau)\, \pi_{\text{time}}(t)$, and evaluate it **only over the 50 legal joint actions**: the three factor logits are summed for each legal a and softmaxed over that set. This keeps the policy factored (three small heads) while making the induced distribution *masking-exact*—it never places probability on an illegal combination. The trunk is a graph network over the memory graph (Section 4), so co-accessed items inform one another's placement rather than being decided independently. Whether this policy class can actually be *fit* to useful accuracy is an empirical question, not a property of the formulation; Section 5 measures the decision such a policy would make, not the policy.

4 THE STOA CONTROL PLANE

STOA is sketched as a control plane above a MemOS-style store, an LMCache-style tiering layer and an M+-style latent path. A graph network over co-access and entity links, as Decima uses one for scheduling [10], feeds a factored policy whose representation, tier and timing heads are masked to the 50 legal joint actions (Section 3), with a budget-conditioned critic. Online actions run on the serving path; offline ones (promotion to latent or parameter forms, summarizing turns into a graph) run in idle windows [2, 8]. Training would combine Belady-oracle imitation [9], process supervision with a learned critic [21] and segment-level advantages over the trajectory segments memory edits induce [23], offline first, in the style of production cache-eviction RL [4]. **None of this machinery is evaluated in this paper.** Section 5 measures the decision it would make, and finds that its value turns less on a better learned ranker than on how far to trust a block it has never seen; we leave the policy as a design.

5 WHAT PLACEMENT HAS TO GET RIGHT

Section 3 frames placement as a learnable budget-constrained MDP. This section reports what we found when we measured that decision on production KV traces, and it is organized around a defect rather than a result, because the defect came first and everything else follows from it: the split-instant protocol, which we used before, makes 84–88% of the achievable benefit unreachable, so a policy measured inside it looks flat however well it ranks (Section 5.2). Moving the decision to each block's arrival puts more of it within reach on the production traces, and where an unseen block ranks against seen ones then matters far more than how unseen blocks are ranked (Section 5.3). We then report what a better ranker buys once the metric is repaired (Section 5.4), where reaction stands (Section 5.6), and, at length because it is the most transferable part, the protocol, including the three claims we retracted along the way (Section 5.7).

5.1 Sources and references

We use **Mooncake** [13], replayed production traces from a general LLM serving stack (conversation, 12,031 requests; toolagent, 23,608), and **kv-cache-tester**, anonymized agentic coding conversations (200 of 739 fetched). Each request lists the hash ids of the KV blocks it touches, so a repeated hash is a genuine reuse. Both sources record wall-clock request timestamps. Block ids in kv-cache-tester are conversation-scoped, so we namespace them before pooling; not doing so invents cross-conversation reuse. Table 2 summarizes both sources.

Two references bound every policy, both billed on the true future: a **hindsight oracle** that ranks and places knowing future access counts, and a **frequency heuristic** that assumes the future looks like the observed prefix. "Headroom" is $(\text{cost}_{\text{heur}} - \text{cost}_{\text{oracle}})/\text{cost}_{\text{heur}}$; "captured" is the fraction of that interval a learned policy recovers. Cost combines latency, money and context tokens with weights $\lambda_L = 0.1$ per ms, $\lambda_C = 1$ per USD and $\lambda_B = 0.0005$ per token, which the artifact documents as illustrative unit-normalising values and has not changed since its first commit.

5.2 The protocol decides what is reachable

A natural way to evaluate placement is to fix a *split instant*: observe the trace up to some point, build features from that prefix, place every block, and bill the result against what the future turns out to contain. It is what we ran in earlier versions of this paper, and it has a structural defect that no aggregate computed inside it can reveal.

The defect is general, not particular to our setup. Whenever a workload admits new items after the split (that is, whenever it is not a closed working set), a split-time policy has no opinion about those items while the hindsight oracle it is scored against ranks them freely. The reported "fraction of headroom captured" then divides by a quantity that is partly unreachable, and the unreachable share grows with the arrival rate. Any study reporting that fraction without bounding it is reporting a number whose denominator it has not characterized.

Feature construction can only describe a block that has been seen. On these traces **45.2% and 46.1% of blocks have no pre-split access at all**, and a policy therefore has no opinion about them, but the hindsight oracle it is scored against ranks them anyway, using knowledge of a future in which they exist. A new block's first access creates it (its KV is computed), a compulsory miss no placement can serve, so only its later accesses are billed. Even so, new blocks carry **84.4% and 87.6%** of the oracle's total advantage, so most of the denominator of every "captured headroom" number we reported was unreachable by construction:

	conversation	toolagent
blocks with no pre-split access	45.2%	46.1%
full hindsight gap	32,002	31,484
reachable gap (visible blocks only)	**4,996**	**3,900**
share no split-time policy can reach	**84.4%**	**87.6%**

It is not an artifact of where we cut, but it needs a long trace. Sweeping the split from 0.3 to 0.7 leaves the unreachable share at **75.0–95.7%** on conversation and **77.4–96.6%** on toolagent, worse at earlier splits, where more of the trace is still to come. Scaling λ_L or λ_B by 0.1 or 10 leaves it at 79.3–97.1%. Trace length matters more: on the first 1,500 requests a split-time policy can reach 38.0% and 57.7% of the advantage, falling to 15.6% and 12.4% on the full traces. The pathology is a property of the protocol on long horizons.

It is a property of arrivals, not of production. Mooncake's third released workload is synthetic (public datasets, Poisson arrivals), and we use it as a control rather than pool it. Only 24.6% of its blocks are new at the midpoint, against 45–46% on the production traces, and the unreachable share falls with them: **54.6%** at the midpoint and **29.4–77.9%** across splits. Within every one of the three traces the share rises monotonically as the split moves earlier; across traces the level differs, production's new blocks carrying more of the oracle's advantage than synthetic ones at a similar new-block share. Even where only 8% of blocks are new, almost a third of the advantage is out of reach.

A policy confined to 12–16% of the prize will look flat however well it ranks, and that is what our earlier "decoupling" result measured. We retract it (Section 5.7).

The diagnostic is one line. Compute the oracle's advantage twice, once with clairvoyance over every item and once restricted to the items the policy could have had an opinion about, and take the ratio, the *reachable share*; the restricted advantage is the *split-time ceiling*, the most any split-time policy can capture. If the share is far from one, the protocol is the finding. It costs a single extra placement pass, it needs no new data, and running it on the first day would have saved us months. We ship it as `attainable_ceiling` in the artifact and recommend it as a standard reporting requirement for placement and admission studies: *state the reachable share alongside any captured fraction.*

5.3 Deciding on arrival, not at a split

Most of the oracle's advantage lives in blocks that do not exist yet when a split-time policy decides. The alternative is an earlier decision: form each block's belief *when it first appears*, from context available at that instant (the shared-prefix session's age, request rate and recency, the block's position within its request, the request's length). Nothing the policy uses reads the future. The arrival population is exactly the blocks first seen at or after the billing split. The model is fitted on blocks first seen before it, each labelled by its reuse within a fixed window that closes before the split. A belief has an order and a magnitude. Three *order* arms rank new blocks by arrival (earliest highest), by a learned score, or at random, and quantile-match that rank onto one multiset, the pre-split blocks' access counts resampled to the number of new blocks, so they differ only in which block receives which value. Two *magnitude* arms give every new block one constant, the mean or the median of that multiset. Two conventions the evaluation must fix are varied: whether placement may also choose a block's representation and timing, as the cost model allows, or the tier only, the one choice physically meaningful for a KV block; and whether a new block whose belief ties a pre-split block's goes behind it or ahead of it. Finally, a *hazard* arm replaces the convention with an estimate: at a pseudo-split halfway through the prefix it measures how often blocks born after that point are reused before the real split, against blocks already present, by count and age. A new block's expected reuse (0.52 and 0.48) exceeds that of a block seen once (0.30 and 0.27), so it is placed between count 2 and count 3, never on a tie. It gives new blocks that belief (hazard new) or every block its estimate (hazard all). Beliefs are causal; the allocation is one static capacity-constrained assignment over the window.

				constant		hazard	
actions, tie		split	order arms	mean	median	new	all
all, behind	conv	15.6	38.2–41.5	53.4	29.4	58.2	69.7
	tool	12.4	38.9–41.5	61.1	29.7	61.1	68.8
all, ahead	conv	15.6	53.1–55.3	53.4	53.4	58.2	69.7
	tool	12.4	56.6–58.1	61.1	56.0	61.1	68.8
tier, behind	conv	20.4	14.3–19.6	39.2	0.0	47.0	56.9
	tool	15.3	15.1–19.4	51.7	0.0	51.7	54.8
tier, ahead	conv	20.4	38.7–42.3	39.2	39.2	47.0	56.9
	tool	15.3	44.3–46.8	51.7	43.3	51.7	54.8

Percent of the full hindsight gap. "split" is the clairvoyant split-time ceiling; the other columns are causal arrival-time arms. Order arms: arrival, learned (AUC 0.56 and 0.61), shuffled.

Ranking unseen blocks above seen-once blocks is what makes arrival pay. Under fixed conventions the arrival arms span 0.0–61.1% of the advantage; with tier-only actions and new blocks behind, the order arms land at the split-time ceiling, 14.3–19.6% against 15.3–20.4%. The reference multiset is itself a convention: matching new blocks onto the pre-split counts minus each block's creation access, the scale the bill uses, puts every order arm at 0.3–1.6 times the split-time ceiling (6.2–21.2%). The prefix's mean reuse and the hazard estimate both place new blocks above seen-once ones; the hazard arm, which needs no tie rule, captures 2.3–4.9 times the matching split-time ceiling (2.3–5.6 when old blocks are re-estimated too), 47.0–69.7% against 12.4–20.4%. On toolagent it coincides with the mean arm, so the estimate's value is the direction it gives, not its precision.

Where an unseen block ranks against seen ones matters most. Moving new blocks ahead of equal-belief seen blocks moves the order arms by 14–29 points (one constant arm by up to 56); ranking new blocks among themselves moves them by at most 5.3. On the production traces arrival order alone predicts reuse with AUC 0.55–0.56 only because early arrivals have longer to be reused (2.3–2.4 times the future accesses of late ones); given every block the same post-arrival horizon, its AUC is 0.46 and 0.48, while the learned ranker's is unchanged.

The synthetic control does not follow. Its split-time ceiling is 45.4% (60.2% tier-only) and every arrival arm, the hazard arms included, stays below it (0.0–40.6%).

Four external traces agree only in part. On Alibaba's Qwen serving traces [18] (four workloads, 16-token blocks) the split protocol leaves 80.6–97.7% of the advantage unreachable, and the hazard arm reaches 1.5–15.2 times the split-time ceiling with all actions. With tier-only actions it reaches 1.9–3.9 times on three and nothing on the fourth (thinking), where the prefix rates seen-once blocks above new ones. The direction the prefix gives is a regularity of these workloads, not a law.

Recency already does this, and migration decides whether anything does. Billed on the same metric from the same starting state with tier-only actions, a multi-tier LRU that moves every accessed block to the fastest tier captures 84.5% and 87.7% of the advantage when moves are free, more than any arrival-time arm replayed online (at most 57.3%); caching a block only on its second access captures −6.2% and −2.8%. Once a move costs one read and one write, LRU falls to −400.5% and −401.4%, and no arrival-time arm, the hazard arm included, beats the static frequency heuristic; the hazard arm keeps 70% of its static gain only if a move costs at most 7.3% and 7.2% of that price. Ranking unseen blocks first is a weaker form of recency, and whether either pays is set by the price of migration, which the static analysis cannot see. We timed that price on one host (an A100 80GB on PCIe Gen4 x16, six runs, a 64 MiB Mooncake block of an 8B-class model): a move between GPU and host memory costs 0.75–1.87 times the read-plus-write price, whether it blocks serving or overlaps it, because the two copy directions did not overlap on this host and a background move took the link time of a load; a demotion to the host's rotational disk costs 0.90–1.46 times. Every measured price is at least 10 times the hazard arm's break-even. The remote tier, which receives most of the hazard arm's online demotions, could not be measured there. Nor does the conclusion depend on how the hierarchy bills a move. Crediting the serving read to a promotion, keeping clean lower copies of immutable blocks, or writing every block through to host memory at creation (the last two without charging the copies' space) leaves LRU at −401.4% to −353.2%, and no production policy pays once a move costs more than 0.28 of the assumed price; most of LRU's moves demote blocks just created, which have no copy below. Only on the synthetic control does a moving policy pay, cache-on-second-hit under the two copy-keeping ledgers (2.6% and 4.5%).

Earlier versions of this experiment were wrong five times, and six of their readings are withdrawn. The first leaked: training labels counted post-split accesses (34.3% and 46.3% of label mass), its train/eval cut differed from its billing split, and its magnitudes were the evaluation blocks' true future counts. The first causal rerun gave pre-split blocks a belief of zero in the arrival arms. The next compared a constant-mean first-come-first-served (FCFS) arm with order arms on the spread multiset, so its order comparison was a magnitude comparison. Every version fixed one tie convention and one action space without saying so. And every version billed a new block's creation as a read from its tier, which inflated the full gap about three times (2.5 with tier-only actions). Withdrawn: that a learned ranker captures less than FCFS, that FCFS captures 87% and 50% of what is reachable, that arrival order is worth 14 points, that FCFS leads the ranker by 2 to 26 points, that this lead reflects exposure time, and that timing recovers nothing on the synthetic control, which the corrected bill has since made true again.

The design implication: build an **admission point**, and *estimate*, rather than assume, where it ranks an unseen block against blocks already seen. Baleen [20] is the one system among those we survey that decides admission rather than eviction, and it optimizes a downstream system cost rather than miss rate.

5.4 What a better ranker buys inside the split protocol

Within the split-instant protocol, does a stronger model convert into a better placement? Each rung is trained on one trace and evaluated on the other, so the AUCs are cross-trace. With ties broken by tier speed and every belief quantile-matched onto one magnitude multiset, the scored blocks' own pre-split counts, so that only the ordering varies:

rung	AUC (conv / tool)	% of ceiling (conv)	(tool)
constant	0.50 / 0.50	−48.6	−60.6
random	0.49 / 0.50	−62.7	−284.0
log(count)	0.64 / 0.71	0.0	0.0
logistic	0.81 / 0.93	+50.7	+55.0
GBDT	0.80 / 0.91	+47.6	+44.9
MLP	0.82 / 0.93	+52.6	+59.5
ceiling	1.00	100	100

Ranking quality converts monotonically on both traces. log(count) reproduces the frequency heuristic exactly, since it ranks by the same counts the magnitudes come from, and each better model recovers more, the best about half of the split-time ceiling. But that ceiling is the 12–16% of Section 5.2: the strongest model gains 8.2% and 7.4% of the oracle's advantage. A previous version of this table placed every rung on the evaluation blocks' true future counts, which no deployed policy has; there every rung fell below the heuristic and conversion failed on one trace.

The magnitude convention moves the result more than ranking does. Running each rung twice, once on the scale its own scores naturally produce and once on the common multiset, *with the ordering identical in both*, gives:

trace	rung	native	common	Δ
conversation	logistic	+74.1	+50.7	23.4
	log(count)	−40.7	0.0	−40.7
	random	−84.5	−62.7	−21.8
toolagent	logistic	+87.8	+55.0	32.7
	log(count)	−25.2	0.0	−25.2
	random	−665.4	−284.0	−381.5

Percent of the split-time ceiling. The two columns differ only in the magnitudes attached to an identical ranking.

The same policy, with the same ordering, scores +87.8% or +55.0% depending on a choice of scale, and across rungs the scale moves the result by **22 to 381 points**, more than any ranking difference above. We report the common-scale column, where only ordering varies, and make the methodological point: **"fraction of headroom captured" is not a quantity until the magnitude convention is stated**, and none of our earlier drafts stated it.

5.5 A measurement that was never a measurement

We also attempted the obvious follow-up (fit a richer per-block model on the agentic source and place with it) and first obtained what looked like a strong positive: +66 to +88% of the headroom captured, reproducing across what we took to be two samples. It

	Mooncake (serving)	kv-cache-tester (agentic)
requests analysed	12,031 / 23,608	4,215 (40 conv.)
blocks touched exactly once	75.8 / 78.5%	15%
mean reuse per block	1.6 / 2.2	26
blocks per request	24 / 17	∼1,900
block size (tokens)	512	64
headroom, heuristic vs. oracle	93%	68–73%

Table 2: The two sources, at the scales actually analysed. Mooncake pairs are conversation / toolagent; its headroom averages the two (93.5% and 91.8%). The agentic column is the first 40 conversation files, except headroom, which is the mean over four randomized draws of 20 and of 40 conversations (Section 5.5). Its descriptive statistics are sample-dependent: the one-time fraction ranges 7.5–21% over 20–200 conversations. An earlier revision took this column's headroom from nested-prefix samples (81–94%), the protocol Section 5.5 retracts.

	sorted-prefix samples (as first reported)	4 random draws mean [observed range]
20 conversations	—	−338% [−858, +78]
40 conversations	+88%	−44% [−166, +66]
70 conversations	+75%	—
100 conversations	−9%	—
150 conversations	−48%	—

Table 3: Headroom captured by a learned placement policy on agentic traces, the same pipeline measured two ways. Left: sorted-prefix samples, which are nested and drawn from a corpus whose per-conversation statistics vary threefold along the file index — the numbers we first reported (regression variant, all features), and their reversal; an earlier revision printed the classifier's +87% at 70. Right: randomized draws at the same sizes, summarized by their observed range — four draws cannot support a parametric interval. The spread is an order of magnitude wider than any effect, so this quantity was never measurable under this protocol. We report it as a cautionary measurement.

was not a result, and the way it failed is worth more than the number would have been.

The two samples were *sorted prefixes* of the conversation corpus, so the smaller was contained in the larger; and file index correlates strongly with conversation shape (mean requests per conversation falls from 140 to 37 between files 1–20 and 71–150). Extending the same prefix to 100 and 150 conversations drove the captured headroom to −9% and −48% — a sign reversal. Re-drawing the samples *at random* rather than as prefixes then showed why: four random draws at 40 conversations land between −166 and +66 points, and at 20 conversations between −858 and +78 (Table 3). The spread dwarfs the effect by an order of magnitude. We quote the observed range rather than a confidence interval on purpose: with four draws a parametric interval rests on a normality assumption these data cannot support, and *captured* is a ratio whose denominator varies per draw. An earlier revision of this paper reported ±126 here, computed with the normal quantile against a standard deviation estimated from four points; the correct $t(3)$ half-width is ±166 on the current artifact. That error was ours. Under a correct sampling protocol there was never a measurement here to report.

We keep this in the paper for two reasons. It is the control that our own protocol (Section 5.7) lacked, and running it costs minutes; and it bounds what the rest of our numbers are worth, since every other result in this section is also a single draw of a parameter we chose.

5.6 Where reaction stands

The alternative to forecasting is to react. On the full Mooncake traces, reactive tiering reaches 75–99% of Belady's hit rate on the toolagent trace and 40–94% on open-ended conversation using LRU, LFU or a fill-once cache alone; admitting the adaptive and learned policies below widens the per-operating-point envelope only to 76–99% and 42–94% (Figure 1).

The interesting question is not the envelope — that is an oracle over policies, not something anyone deploys — but whether any *single* policy attains it, and whether the adaptive designs built for exactly this situation beat plain rules. We therefore run the family: **ARC** [11], which shifts a recency/frequency split on ghost hits; **LeCaR** [17] and **CACHEUS** [14], which replace ARC's arithmetic update with regret-based multiplicative weights over two experts; and the learned **LRB**, which is reported separately below because its numbers carry an implementation band the others do not.

trace	tier	best pure	ARC	LeCaR	CACHEUS
conversation	2%	39.8	29.4	**42.3**	35.9
	5%	53.3	**57.9**	56.8	53.5
	10%	76.0	**77.5**	77.2	75.3
	20%	**94.1**	87.4	93.9	88.8
toolagent	2%	74.9	70.7	**75.8**	73.9
	5%	81.3	**83.6**	83.0	82.9
	10%	91.9	92.3	**92.9**	91.8
	20%	99.3	96.3	**99.3**	97.6

Percent of Belady's hit rate. "Best pure" is the better of LRU, LFU and a fill-once cache at that operating point.

Across eight operating points and four designs, **the best adaptive or learned policy beats the best pure rule by at most 4.6 points of Belady**, and at the loosest tier by at most 0.1. Which design wins also moves: LeCaR at the tightest tier on both traces and ARC at 5%; above that, ARC and then plain LRU on conversation, LeCaR on toolagent. That is the substantive finding — not that these policies fail, but that on KV-block workloads the whole adaptive-replacement family collapses into a band a few points wide, well short of the oracle at the small tiers where the decision actually matters.

The diagnostics say why, and the reason is specific to KV workloads. Every policy in this family adapts on a *ghost hit* (a miss on a block it recently evicted), whether it updates a split (ARC) or an expert weight (LeCaR, CACHEUS). But 76–78% of blocks in these traces are never reused at all, so most evictions are correct and produce no signal. Measured, ARC receives an adaptation signal

on **3.5–7.3% of misses**; the regret-based policies read the same channel, which is why they land in the same band rather than above it. What signal it does get is not merely sparse but, at the smallest tier, directionally wrong: at a 2% tier ARC drives p to 90% recency on both traces, while the rule that actually wins there is frequency. A workload whose blocks are mostly singletons starves the very mechanism adaptive replacement runs on.

And the learned cache policy loses to both — but only as a range. The natural learned baseline is LRB [16], which learns a relaxed-Belady objective from a far richer feature set than our four trace-local statistics and won on production CDN traces. We reimplement it: GBDT regression on log time-to-next-request over inter-access deltas and exponentially decayed counters, sampled eviction, online retraining from a sliding memory window, right-censored labels.

Our first version of it measured something else entirely. The training buffer was capped by truncating to its tail; labelled rows arrive one at a time as blocks are re-accessed while censored rows arrive in one contiguous burst per checkpoint, so from the third retrain onward every training label was identical, the regressor returned a constant, and eviction degenerated to a uniform draw over the sampled candidates. We report the repaired numbers and, because two defensible repairs disagree, we report them as a band:

trace	tier	LRB (repaired)	ARC	below ARC?
conversation	2%	27.6–29.2	29.4	yes
	5%	30.9–32.2	57.9	yes
	10%	45.0–45.9	77.5	yes
	20%	75.8–76.5	87.4	yes
toolagent	2%	68.8–69.0	70.7	yes
	5%	71.1–71.4	83.6	yes
	10%	80.9–83.0	92.3	yes
	20%	94.6–94.9	96.3	yes

Percent of Belady. The band spans two repairs of the truncation bug: subsampling all accumulated rows, and dropping rows older than the memory window first. We adopt the second, which is what a "sliding memory window" means.

What the band supports. The two repairs agree to within 2.1 points at every operating point, and **LRB is below ARC at all eight operating points under both.** That ordering is the claim we make; the numbers are context for it. Both repairs and the retracted behaviour are selectable in the artifact, so the band is regenerated rather than reported. We do not claim the band bounds every reasonable reimplementation: a third repair produced in an AI-agent review we ran (see the Acknowledgment) reported a higher figure, and since its code is not ours to ship, we neither reproduce nor rely on it.

The mechanism appears to be the one that limits ARC, seen from the other side. LRB learns from labels the trace supplies when a sampled block is requested again; a block that never returns yields a *right-censored* row, which says only "not within the window" and nothing about when. At the default window **83–88% of LRB's training rows are censored**, and 88–91% at a shorter one. A population dominated by singletons starves the ghost-hit channel that adaptive replacement runs on and the regression target that learned replacement runs on, for the same underlying reason.

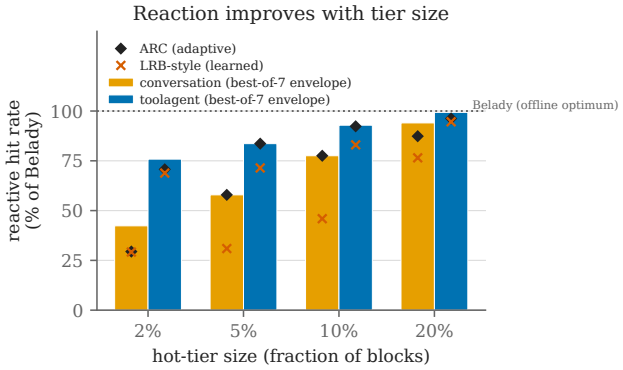


Figure 1: Reactive tiering as a fraction of Belady's hit rate, full Mooncake traces. Bars are a per-operating-point best-of-seven envelope, an oracle over policies rather than a policy. Diamonds are ARC, which meets the envelope at 5% on both traces and at 10% on conversation but falls short elsewhere; crosses are the LRB-style learned policy, which is below ARC everywhere.

Three caveats bound this. It is our reimplementation of the published mechanism, not the authors' artifact. KV blocks are uniform-size, so byte miss ratio collapses to object miss ratio and LRB loses one of its structural advantages over LRU. And our first hyperparameter sweep ran on the *broken* implementation, so it tested the robustness of random eviction. Rerun on the repaired one at a 10% tier (six settings of window, sample size and trees per trace), LRB spans 41.9–61.8% of Belady on conversation and 77.6–86.4% on toolagent, below ARC's 77.5% and 92.3% at every setting; the band is too wide to quote one LRB number, but not wide enough to reach ARC.

The eviction signal also inverts by source. On the agent traces LRU reaches 0.83–0.96 of Belady at a 5% tier while LFU stays near 0.29–0.31; on Mooncake the ordering reverses below the working-set knee. An eviction policy tuned on one source is a poor default for the other — and, as both the ARC and the LRB result show, reaching for a more sophisticated policy does not by itself resolve the choice.

5.7 What the measurement required

Every positive result we obtained before the final protocol was an artifact, and each looked reasonable: a flat captured fraction, a plausible interval, a learned policy landing below ARC. Two of the twenty-seven defects in Table 4 were found by an adversarial review we ran with AI reviewer agents (see the Acknowledgment), five by an audit and the rest by re-verification passes, all after the paper had passed every check it defined for itself. The artifact's lab notebook records each one with its fix.

Two of them changed what this paper claims. A headroom taken at 1,500 requests moved by full length from 86.4% to 91.8% on toolagent and from 82.9% to 93.5% on conversation, and a protocol evaluated on short traces would have found far more reachable than the full traces show (Section 5.2). And the conventions of Section 5.3, each fixed for convenience, moved one arm by up to

mechanism (count)	what happened	control now in the harness
Reference-frame mismatch (6)	a split over access-event indices that fell past the trace's end; ties broken by label (AUC 0.51 to 0.94); an oracle inheriting the heuristic's order; features built at one split instant, costs billed at another	one split instant for features, labels and bill; frames compared explicitly
Retracted claims (3)	retracted: a "+66–88% captured" agentic headline that sampling showed was never measurable; the *decoupling* claim, an artifact of an unreachable denominator; an LRB column that measured random eviction because its training buffer was truncated	randomized draws; the reachable share beside any captured fraction; LRB fails on zero label variance
Free-parameter extrapolation (3)	curves read from one point: a cache size, a trace length, a conversation count (+88% reversing to −48%)	every convenience parameter swept
Believing a report (1)	an edit script printed success after matching nothing	edits asserted to land
Stale derived or parallel surfaces (3)	artifacts older than the code that made them; figures fifteen days older than their data; a notebook stating a falsified claim	content-hash provenance; figure and PDF manifests; banners on records
Unchecked summaries (1)	a sentence misquoted a checked band's width	spans and maxima asserted; withdrawn values must be absent
Unverified interventions (1)	a mutation whose target string was absent "passed"	interventions asserted to apply; guards tested both ways
Emphasis without verification (1)	the headline share sat in no artifact and mixed two references (97% to 95%, then 84–88%)	every emphasised number asserted
Leaks and unstated conventions (8)	in the arrival experiment: post-split labels; a cut unlike the bill; magnitudes from evaluation blocks' futures; zero beliefs for pre-split blocks; a mixed-scale ceiling; an order control that also changed magnitude; unstated tie and action-space conventions (up to 56 points); creations billed as reads (the gap inflated about three times)	a causal module with an AST taint test; conventions swept; billable access defined in one place

Table 4: The twenty-seven measurement defects, by mechanism. Each produced a plausible number that matched its artifact.

56 points. Three controls would have caught most of the catalogue on the first day: compute the reachable share beside any captured fraction; treat every convenience choice as a variable to sweep; and, before reporting a null, compute the sample size that would have resolved it, because "we could not tell" and "there is nothing there" are different claims. The harness ships with these wired in, and with a checker that re-derives every result in this paper from its artifact and fails on drift.

5.8 Limitations

No serving deployment was measured, only block moves on one host (Section 5.3), so any cost-expressed quantity should be read as a range. We tightened what that range means. Our first sensitivity analysis scaled every tier latency by a common factor, which moves the absolute scale and leaves the *ratios* between tiers untouched — and the ratios are what a placement policy responds to, so it never tested the parameter that mattered. We therefore derive the table instead: KV bytes per block follow exactly from a model's attention shape ($2 \times$ layers $\times$ kv_heads $\times$ head_dim $\times$ dtype per token), and per-tier latency is that divided by a declared device-class bandwidth plus a fixed overhead. Sweeping three attention shapes against five device classes puts headroom at **55–98%**, which contains the 92% our hand-written table gives — the headroom conclusion survives (we did not rerun the unreachable share or the arrival arms under the derived tables), but two things about the table do not. Its GPU:CPU latency ratio is 1:10 where every device class but the deliberately flat one gives 1:80 to 1:250 (our A100 host measures 1:58), an order of magnitude in the ratio the policy is most sensitive to; and **6 of the 15 configurations reorder the hierarchy** so that latency order and capacity order disagree, which our fixed per-tier capacity table cannot express at all. Calibrating against a live deployment remains the open gate; this is the honest intermediate, not a substitute. The *timing* and *representation* axes of Section 3 are chosen by the simulator from placeholder cost tables but carry no signal from the traces, which is why Section 5.3 also reports tier-only placement. A representation pilot over three textual fidelities is in the artifact, but it depends on a paid hosted model, and this paper reports only what the artifact regenerates without one. Both sources are KV-block hash streams from transformer prefix caching, so they are independent in provenance but not in mechanism. Alibaba's four traces (16-token blocks) and Mooncake's synthetic control broaden the set, but we still cannot separate workload character from vendor, date, or block size. Sampling is not randomized, every placement result is a single deterministic run, and no confidence interval is reported for them, though trace length, split instant, cost weights, tie convention and action space are now swept rather than fixed. The arrival-time arms form beliefs causally but are allocated capacity in one static assignment, which is neither a bound nor an admission policy: replayed online they need nearly free migration to keep their gain (above). The hazard estimate fixes one pseudo-split, its count bins and its age quartiles, and applies counts from half the prefix to counts over the whole of it; rescaling for that moves the hazard arm by at most 0.6 points on the production traces. Of the learned and adaptive cache literature we evaluate ARC [11], LeCaR [17], CACHEUS [14] and a reimplementation of LRB [16]; Hawkeye, Mockingjay and Baleen are positioned in Section 7 but not run, the first two because a KV block hash carries no analogue of the program counter their predictors are indexed by. Our LRB is the published mechanism rebuilt from the paper, not the authors' artifact (as are our LeCaR and CACHEUS, whose CR-LFU expert we approximate by LFU), and the uniform block size of both sources removes one of LRB's advantages over LRU, so the underperformance we report should be read as a statement about this workload rather than about these designs.

6 OPEN PROBLEMS

Our result closes one question and sharpens several others.

What should an admission point believe about a block it has never seen? Section 5.3 shows that deciding at arrival puts more of the oracle's advantage within reach, and that the result then turns on

one decision: where a block never seen ranks against blocks seen once. The prefix gives its direction; learning it per session or per request, and admitting online at a migration cost low enough to keep the gain, which our replay did not find under a read-plus-write price, is the obvious next model. Ranking new blocks against each other is a second-order question, and comparing rules on it needs an evaluation that gives every block the same exposure, a rolling window or a fixed post-arrival horizon, because in a finite trace early arrivals have more time to be reused. Whatever such an evaluation leaves unclaimed has to come from something the per-item formulation does not express: admission under contention, coordinated eviction across concurrently-live sessions, or migration timed against idle windows.

Calibration, and what a mis-specified cost model hides. Our tables are placeholders, and we saw concretely how misleading that is: under them the per-item optimal action is identical for every reused block, which collapses a three-axis decision into a one-dimensional sort and makes every policy look alike. Fitting the tables to a real vLLM+LMCache deployment — the only GPU-gated step in this work — is a precondition for any claim about the representation and timing axes, which our traces do not exercise at all.

Aggregate working set, not per-request footprint. Reactive sufficiency is governed by how much of the *concurrently live* working set a tier holds, not by one request's block count: holding tier size fixed in absolute blocks and raising the number of pooled conversations still degrades the ratio. Characterizing that surface — tier size against concurrency, across sources — would give an operator a sizing rule, which is the most directly actionable thing this line of work could produce.

Composing placement with retrieval. Placement sets latency and cost; retrieval sets which facts reach the model. The open problem is the composition — a retriever running over a tiered store whose hot set a scheduler maintains, where a retrieval miss on a cold tier costs latency rather than accuracy. Neither literature evaluates that joint object, and the KV traces here cannot: they record block reuse, not which facts a query needed.

Sampling as a first-class experimental variable. Three of the defects in Section 5.7 were extrapolations from a single value of a parameter fixed for convenience, and the most damaging reversed a headline claim's sign. For trace-driven work we would now require of ourselves, and suggest for the area: randomized rather than prefix sampling, several disjoint draws with intervals, and a reported sweep over every quantity the design fixes — trace length, pool size, tier size, split fraction — before any claim that depends on one of them.

Safety and governance. A memory written by tools and other agents is an attack surface, and provenance must survive consolidation so a fact can be traced, invalidated, or unlearned. A budget-constrained scheduler is a natural enforcement point; doing that without giving back the frontier is unexplored.

7 RELATED WORK

We organize prior work by the three axes STOA unifies; Table 1 summarizes which axis each line decides and by what means.

Representation and memory-OS. MemOS [7] and MemGPT [12] treat memory as an operating-system problem, exposing abstractions (MemCube, virtual context) and primitives to migrate items between plaintext, activation, and parameter forms; M+ [19] adds a co-trained latent memory. These provide the *mechanism* to change representation and tier but decide *when* by heuristic; STOA frames the learned scheduler they lack; we do not build it.

Content and RL-based management. Memory-R1 [22] and Memory-as-Action [23] learn, with RL, *what* to add, update, or delete and how to edit context, and the latter handles the trajectory-fracturing that edits cause. But they operate over a single fixed representation and never choose a physical tier or schedule offline consolidation—orthogonal to, and composable with, STOA.

Systems tiering and ML-for-systems. Serving systems already make prefix KV blocks reusable: PagedAttention [6] manages them in pages that requests can share, SGLang [25] reuses them across requests through a prefix tree, and CachedAttention [3] and Mooncake [13] keep them across a hierarchy of storage tiers, with hand-designed policies for what stays where; our traces come from the last of these. LMCache [1] tiers KV-cache bytes to cut miss-rate; H2O [24] evicts by attention score. The learnability of such decisions is established: PARROT [9] imitates a Belady oracle for cache replacement, Cold-RL [4] does eviction with offline RL in production, and Decima [10] schedules clusters with a GNN policy. STOA would inherit these techniques but change the objective from miss-rate to downstream *task utility*, and the action from a single axis to the joint representation × tier × timing decision.

Offline consolidation. Sleep-time compute [8] and LightMem [2] show the value of precomputing and consolidating memory offline, but treat it as a global mode rather than a per-item, budget-aware timing decision, the third axis of STOA's frame.

Learned and adaptive caching. Two lines bear directly on our results and we position against them explicitly, because our findings reproduce rather than extend them. **LRB** [16] learns a relaxed-Belady objective for CDN caching from a feature vector built out of the deltas between an object's past request times — the same family as the session recency and rate features our arrival-time ranker uses. Timing as a cache-scheduler input is therefore established practice; what we add is evidence about *when* a placement decision is taken, not a new signal to take it with. **ARC** [11] adaptively balances recency against frequency precisely because neither dominates across workloads; the recency-versus-frequency inversion we measure between our two sources (Section 5.6) is that observation restated on KV blocks, and an adaptive policy of the ARC family is the baseline a deployment should reach for rather than either pure rule.

The rest of the learned-cache literature divides by what it learns *from* and what it decides, and the division explains why our result is not a contradiction of it. LeCaR [17] and CACHEUS [14] learn no per-item model at all: they run experts (LRU/LFU, and in CACHEUS scan- and churn-resistant variants) and update expert weights from regret observed on a ghost list. That is the same evidence channel ARC uses, and Section 5.6 measures how thin it is here — 3.5–7.3% of misses carry any signal — so the family inherits the limitation rather than escaping it. We ran both to check,

rather than leaving it as a prediction, and both land in the same few-point band as ARC. LeCaR is nonetheless the best single policy at the tightest tier on both traces, which is the one place the regret formulation visibly pays. Hawkeye [5] and Mockingjay [15] are the closest antecedents to what we attempted: both supervise a predictor with Belady replayed over history, Hawkeye as a binary cache-friendly/averse label and Mockingjay as a multiclass reuse-distance estimate. They operate at CPU last-level-cache granularity, where a program counter identifies a reuse pattern; a KV block hash carries no analogous instruction identity, which is the structural reason their signal does not simply port. Baleen [20] is the one that decides admission and prefetching rather than eviction, and the one that optimizes a downstream system cost (disk-head time) instead of miss rate — the closest precedent for the multi-objective reward of Section 3, and the direction we think is right even though we do not evaluate it here.

Of these we evaluate LRB, LeCaR and CACHEUS (Section 5.6); Hawkeye, Mockingjay and Baleen are positioned but not run.

In short, each line optimizes one axis and freezes the others, and none decides representation, tier, *and* timing jointly under a shared budget (Table 1). Our contribution is not a better policy but a measurement: on production KV traces a split-instant protocol leaves most of a hindsight oracle's advantage out of a split-time policy's reach, deciding at each block's arrival brings much of it back into reach in a static analysis, and whether that or plain recency pays online is set by the price of migration.

8 CONCLUSION

For every item an agent remembers, three coupled questions (in what representation, on which tier, and when to materialize it) are today answered separately by three research communities. We offered a single budget-constrained formulation as a frame and measured one of its axes, placement across tiers, on production KV traces.

The finding is about what an evaluation lets a policy see, and it is first a finding about evaluation. Evaluated at a split instant, as is natural, at least three-quarters of a hindsight oracle's advantage is out of reach of any policy that places the blocks present at the split, for every split we tried and under tenfold changes to the cost weights, provided the trace is long (on its first 1,500 requests far more is reachable); inside that protocol a better ranker helps monotonically, but over a sliver. Deciding at each block's arrival puts more within reach on both production traces when a block never seen is ranked above blocks seen once, as the prefix says it should be: 2.3–4.9 times the split-time ceiling in a static analysis. Plain recency, billed the same way, captures more when moving blocks is free, and once a move is priced none of the moving policies we replay beats the static heuristic: what the protocol hides is recency and the price of migration. Reactive tiering, forecasting nothing, already delivers much of the achievable hit rate, and the adaptive-replacement family adds little to it, because the signal it adapts on is scarce when most blocks are never reused.

We arrived here only after removing twenty-seven measurement defects, each of which had produced a plausible result, including leaks and an unfair control in our own arrival-time experiment that were found after every number check had passed. We report their mechanisms and the controls that catch them, because the same traps are open to anyone measuring a memory scheduler. The next steps are an admission rule that learns the belief it gives unseen blocks, an evaluation with equal exposure per block, a cost model calibrated against real tiers, and the habit of asking what a reported fraction's denominator could have been.

ACKNOWLEDGMENTS

Use of AI-generated content. An AI assistant (Anthropic's Claude, Opus 5 and Opus 5.5 models, used through the Claude Code agentic environment) was used extensively throughout this work. It wrote most of the code in the released artifact, designed and ran most of the experiments and analyses, drafted and revised most of the text, and generated the figures from the stored results; its contributions are recorded commit by commit in the artifact's history. The "adversarial review", the audit and the re-verification passes referred to in Section 5.7 were internal exercises in which AI agents were prompted to act as reviewers; none was peer review, and one figure the review produced (a third LRB repair) is neither reproduced nor relied on here. Every result in this paper is re-derived from the released artifact by a checker shipped with it; a few historical figures (what an earlier version reported) come from the artifact's lab notebook. The initial concept and research idea were proposed by the author. The author takes full responsibility for the content of this paper.

REFERENCES

[1] Yihua Cheng, Yuhan Liu, et al. 2025. LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. *arXiv* (2025). arXiv:2510.09665

[2] Jizhan Fang et al. 2025. LightMem: Lightweight and Efficient Memory-Augmented Generation. *arXiv (ICLR 2026)* (2025). arXiv:2510.18866

[3] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, et al. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *USENIX Annual Technical Conference (ATC)*. arXiv:2403.19708

[4] Aayush Gupta and Arpit Bhayani. 2025. Cold-RL: Learning Cache Eviction with Offline Reinforcement Learning for NGINX. *arXiv* (2025). arXiv:2508.12485

[5] Akanksha Jain and Calvin Lin. 2016. Back to the Future: Leveraging Belady's Algorithm for Improved Cache Replacement. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 78–89. https://doi.org/10.1109/ISCA.2016.17

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *29th ACM Symposium on Operating Systems Principles (SOSP)*. arXiv:2309.06180

[7] Zhiyu Li et al. 2025. MemOS: A Memory OS for AI System. *arXiv* (2025). arXiv:2507.03724

[8] Kevin Lin, Charlie Snell, Yu Wang, Charles Packer, Sarah Wooders, Ion Stoica, and Joseph E. Gonzalez. 2025. Sleep-time Compute: Beyond Inference Scaling at Test-time. *arXiv* (2025). arXiv:2504.13171

[9] Evan Zheran Liu, Milad Hashemi, Kevin Swersky, Parthasarathy Ranganathan, and Junwhan Ahn. 2020. An Imitation Learning Approach for Cache Replacement. *ICML* (2020). arXiv:2006.16239

[10] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. 2019. Learning Scheduling Algorithms for Data Processing Clusters. *SIGCOMM* (2019). arXiv:1810.01963

[11] Nimrod Megiddo and Dharmendra S. Modha. 2003. ARC: A Self-Tuning, Low Overhead Replacement Cache. In *2nd USENIX Conference on File and Storage Technologies (FAST)*. https://www.usenix.org/conference/fast-03/arc-self-tuning-low-overhead-replacement-cache

[12] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv* (2023). arXiv:2310.08560

[13] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST)*. https://www.usenix.org/

conference/fast25/presentation/qin Traces: github.com/kvcache-ai/Mooncake, FAST25-release.

[14] Liana V. Rodriguez, Farzana Beente Yusuf, Steven Lyons, Eysler Paz, Raju Rangaswami, Jason Liu, Ming Zhao, and Giri Narasimhan. 2021. Learning Cache Replacement with CACHEUS. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. USENIX Association, 341–354. https://www.usenix.org/conference/fast21/presentation/rodriguez

[15] Ishan Shah, Akanksha Jain, and Calvin Lin. 2022. Effective Mimicry of Belady's MIN Policy. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 558–572. https://doi.org/10.1109/HPCA53966.2022.00048

[16] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. 2020. Learning Relaxed Belady for Content Distribution Network Caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 529–544. https://www.usenix.org/conference/nsdi20/presentation/song

[17] Giuseppe Vietri, Liana V. Rodriguez, Wendy A. Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. 2018. Driving Cache Replacement with ML-based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*. USENIX Association, Boston, MA. https://www.usenix.org/conference/hotstorage18/presentation/vietri

[18] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. 2025. KVCache Cache in the Wild: Characterizing and Optimizing KVCache Cache at a Large Cloud Provider. In *USENIX Annual Technical Conference (ATC)*. arXiv:2506.02634

[19] Yu Wang, Dmitry Krotov, Yuanzhe Hu, et al. 2025. M+: Extending MemoryLLM with Scalable Long-Term Memory. *ICML* (2025). arXiv:2502.00592

[20] Daniel Lin-Kit Wong, Hao Wu, Carson Molder, Sathya Gunasekar, Jimmy Lu, Snehal Khandkar, Abhinav Sharma, Daniel S. Berger, Nathan Beckmann, and Gregory R. Ganger. 2024. Baleen: ML Admission and Prefetching for Flash Caches. In *22nd USENIX Conference on File and Storage Technologies (FAST 24)*. USENIX Association, Santa Clara, CA, 347–371. https://www.usenix.org/conference/fast24/presentation/wong

[21] Guangzhi Xiong, Qiao Jin, Xiao Wang, et al. 2025. RAG-Gym: Optimizing Reasoning and Search Agents with Process Supervision. *arXiv:2502.13957v1* (2025). arXiv:2502.13957

[22] Sikuan Yan et al. 2025. Memory-R1: Enhancing LLM Agents to Manage and Utilize Memories via Reinforcement Learning. *arXiv* (2025). arXiv:2508.19828

[23] Yuxiang Zhang et al. 2025. Memory as Action: Autonomous Context Curation for Long-Horizon Agentic Tasks. *arXiv* (2025). arXiv:2510.12635

[24] Zhenyu Zhang et al. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *NeurIPS* (2023). arXiv:2306.14048

[25] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, et al. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2312.07104